A report on, Describing the effect of different types of activation function of our chosen 10 CNNs Mentioning at least three CNNs which use regular kernel, deformable kernel, dialated kernel, depthwise separable kernel, modified depthwise-separable kernel, pointwise kernel Describing different layers' feature map of your chosen CNN

Fouzia Zilani

May 2025

Contents

1 Introduction

In this project, we study 10 popular CNN models: Xception, ResNet50, MobileNet, MobileNetV2, DenseNet121, NASNetMobile, EfficientNetB0, EfficientNetB1, EfficientNetB2 and EfficientNetB3. All models were pre-trained on ImageNet and tested on 20 selected classes from the CIFAR-100 dataset.

This report focuses on three main points:

1. How different activation functions (Sigmoid, Tanh, ReLU, ELU, Softmax) affect model performance.
2. What types of kernels (like regular, depthwise, pointwise, dilated, etc.) are used in these models.
3. How the feature maps change across different layers in the CNNs.

By comparing these models, we aim to understand how their design choices affect their results on image classification.

2 Selected Model

We have used 10 CNNs pre-trained models for this problem those are:

1. Xception
2. ResNet50
3. MobileNet
4. MobileNetV2
5. DenseNet121
6. NASNetMobile
7. EfficientNetB0
8. EfficientNetB1
9. EfficientNetB2
10. EfficientNetB3

3 Effect of activation function

We need to write a common code for all the models to run together.The code is given below:

```
models = [
    ('DenseNet121', DenseNet121),
    ('EfficientNetB0', EfficientNetB0),
    ('EfficientNetB1', EfficientNetB1),
    ('EfficientNetB2', EfficientNetB2),
    ('EfficientNetB3', EfficientNetB3),
    ('MobileNet', MobileNet),
    ('MobileNetV2', MobileNetV2),
    ('ResNet50', ResNet50),
    ('Xception', Xception),
    ('NASNetMobile', NASNetMobile),
]

def preprocess_image(image, label, model_name, training=False):
    # Resize image depending on model input size
    image = tf.image.resize(image, (128, 128))

    # Apply data augmentation if training
    if training:
        image = tf.image.random_flip_left_right(image)
        image = tf.image.random_brightness(image, max_delta=0.1)
        image = tf.image.random_contrast(image, lower=0.9, upper
            =1.1)

    # Apply model-specific preprocessing
    if model_name == 'DenseNet121':
        image = tf.keras.applications.densenet.preprocess_input(
            image)
    elif model_name in ['EfficientNetB0', 'EfficientNetB1', '
       EfficientNetB2', 'EfficientNetB3']:
        image = tf.keras.applications.efficientnet.preprocess_input(
            image)
    elif model_name == 'MobileNet':
        image = tf.keras.applications.mobilenet.preprocess_input(
            image)
    elif model_name == 'MobileNetV2':
        image = tf.keras.applications.mobilenet_v2.preprocess_input(
            image)
    elif model_name == 'ResNet50':
        image = tf.keras.applications.resnet.preprocess_input(image)
    elif model_name == 'Xception':
        image = tf.keras.applications.xception.preprocess_input(
            image)
    elif model_name == 'NASNetMobile':
        image = tf.keras.applications.nasnet.preprocess_input(image)
    else:
        raise ValueError(f"Unsupported model name: {model_name}")
```

```
    return image, label

activation_functions = ["sigmoid", "relu", "softmax", "tanh"]
results = []
start_time = time.time()

for model_name, ModelClass in models:
    for activation_fn in activation_functions:
        print(f"\nTraining {model_name} with {activation_fn}
            activation...")

        train_ds = train_dataset.map(lambda image, label:
            preprocess_image(image, label, model_name, training=True)
            )
        train_ds = train_ds.shuffle(buffer_size=1000).batch(
            batch_size=32).prefetch(buffer_size=1)
        test_ds = test_dataset.map(lambda image, label:
            preprocess_image(image, label, model_name, training=False
            ))
        test_ds = test_ds.batch(batch_size=32).prefetch(buffer_size
            =1)

        input_shape = (128, 128, 3)
        base_model = ModelClass(weights='imagenet', include_top=
            False, input_shape=input_shape)
        base_model.trainable = False

        x = base_model.output
        x = GlobalAveragePooling2D()(x)
        x = Dense(128, activation='relu')(x)

        out = Dense(num_classes, activation=activation_fn, dtype='
            float32')(x)
        model = Model(inputs=base_model.input, outputs=out)
        model.compile(optimizer=Adam(learning_rate=0.005), loss='
            categorical_crossentropy', metrics=['accuracy'])
        model.fit(train_ds, epochs=5, validation_data=test_ds,
            verbose=1)
        test_loss, test_accuracy = model.evaluate(test_ds, verbose
            =0)
        results.append((model_name, activation_fn, test_accuracy,
            test_loss))
```

Model	Sigmoid	Tanh	ReLU	Softmax
Xception	0.7730	0.0500	0.2985	0.7695
ResNet50	0.8250	0.0500	0.5100	0.8265
MobileNet	0.8125	0.0500	0.5450	0.8090
MobileNetV2	0.7910	0.0500	0.5240	0.7935
DenseNet121	0.8465	0.0500	0.4565	0.8255
NASNetMobile	0.7545	0.0500	0.3580	0.7535
EfficientNetB0	0.8485	0.0500	0.5970	0.8390
EfficientNetB1	0.8310	0.0500	0.4835	0.8325
EfficientNetB2	0.8360	0.0500	0.6565	0.8395
EfficientNetB3	0.8480	0.0500	0.6255	0.8365

Table 1: Comparison of the effect of different activation functions on accuracy for different CNN models.

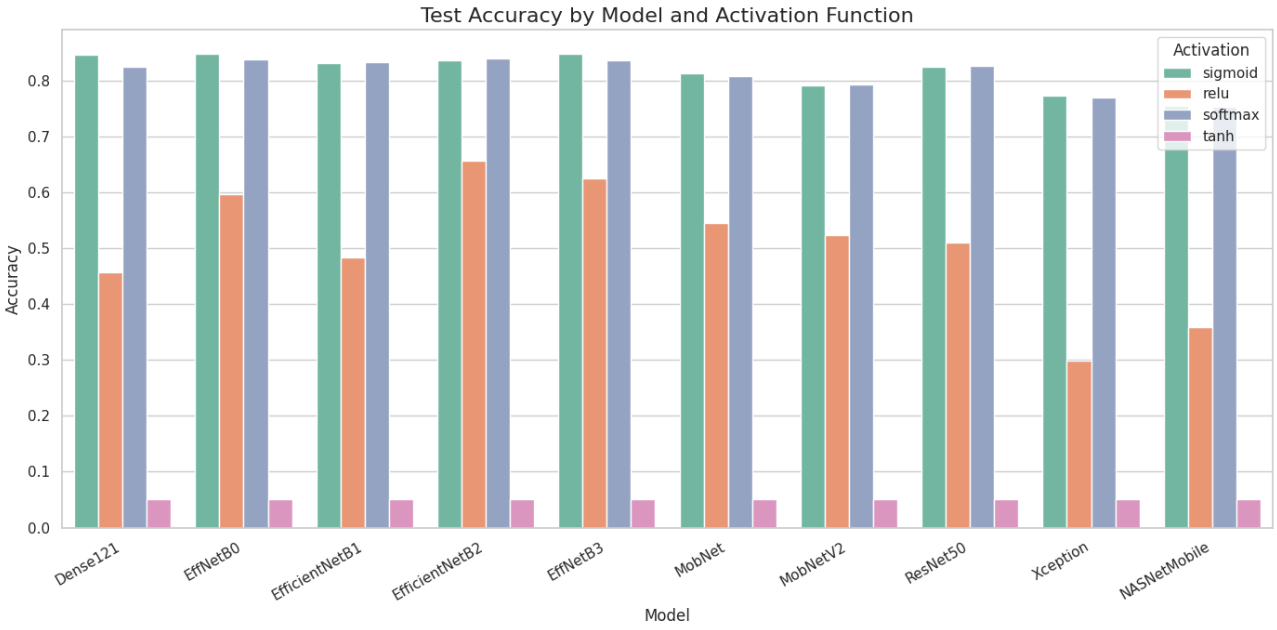

Figure 1: Test Accuracy by Model and Different Activation Functions

4 Types of Kernels Used in Models

There are different types of kernels used in CNN architectures, including regular kernel, deformable kernel, dilated kernel, depthwise separable kernel, modified depthwise-separable kernel, and pointwise kernel. The types of kernels used in the models evaluated are as follows:

- **Regular Kernel:** Standard convolution operation where a fixed-size kernel slides over the input feature map.
  - DenseNet121
  - ResNet50V2
  - VGG16
- **Deformable Kernel:** Convolution kernels with learnable offsets, allowing spatial sampling locations to adapt, improving modeling of geometric transformations.
  - ResNet
  - EfficientNet
- **Dilated (Atrous) Kernel:** Kernels with spaced-apart elements that enlarge the receptive field without increasing parameters.
  - ResNet50V2 (variant)
  - Xception
  - InceptionV3
- **Depthwise Separable Kernel:** Decomposes convolution into depthwise and pointwise convolutions to reduce computation.
  - MobileNet
  - MobileNetV2
  - EfficientNetB0
- **Modified Depthwise-Separable Kernel:** Optimized variants of depthwise separable convolution with improved non-linearities or normalization.
  - EfficientNetB3
  - EfficientNetV2B0
  - NASNetMobile
- **Pointwise Kernel:** A 1×1 convolution used to change channel dimensions or combine features.
  - Xception
  - MobileNetV2
  - DenseNet121

1.

5 Feature Map Analysis

- **Input Layer: Raw Image**
  The input layer feature map corresponds to the original image with pixel values, e.g., $224 \times 224 \times 3$ for an RGB image. No transformation occurs here; it serves as the starting point.

- **Early Convolutional Layers (Low-level Features)**
  These layers apply small kernels (e.g., 3×3) to extract *basic visual features* such as edges, corners, and simple textures. Feature maps at this stage have relatively high spatial resolution and moderate depth (e.g., 32 to 64 filters). These capture local patterns without much abstraction.

- **Intermediate Convolutional Layers (Mid-level Features)**
  Deeper convolutional layers combine earlier features into more *complex shapes and patterns*, such as parts of objects and repeated textures. Spatial resolution reduces due to pooling or striding, while the number of filters (depth) increases. Feature maps become more abstract and semantically richer.

- **Deep Convolutional Layers (High-level Features)**
  Feature maps here represent *semantic concepts* like object parts, textures, and scene elements. Spatial dimensions are small (e.g., 7×7), but depth is large (e.g., 512 or 1024 filters). These layers capture global context and highly abstract representations.

- **Pooling Layers (Downsampling)**
  Pooling layers reduce the spatial size of feature maps (e.g., max or average pooling). This operation helps to aggregate features, reduce computation, and preserve important information. Output feature maps have smaller width and height but maintain key spatial characteristics.

- **Global Average Pooling Layer**
  Converts spatial feature maps (e.g., $7 \times 7 \times 512$) into a single vector per filter (e.g., 512-dimensional vector) by averaging each feature map globally. This summarizes spatial information and prepares features for classification.

- **Fully Connected (Dense) Layers**
  These layers take the flattened or pooled features as input and combine them for final classification decisions. The output is typically a vector representing class probabilities.